

BIG DATA ANALYTICS

Distributed Multi-Model Analytics for E-Commerce Data

Report | June 2026 |

ID: 101230

Names: Shyaka Hamza

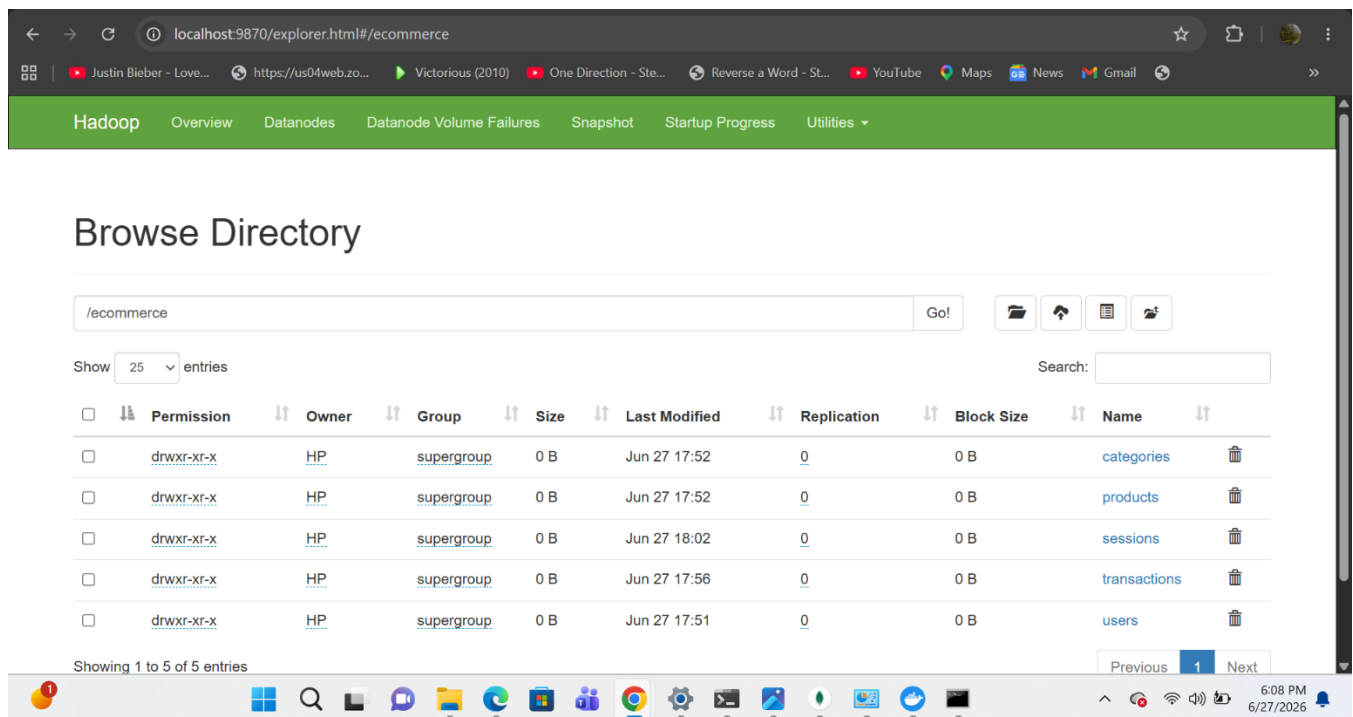
Github Link: <https://github.com/shyakahamza/ecommerce-big-data-analytics.git>

1. System Architecture Overview

This project implements a distributed analytics system that uses MongoDB, HBase, and Apache Spark together — each technology doing what it does best. Raw data lives in HDFS, Spark processes it, and results go into MongoDB (for document queries) and HBase (for time-series session lookups).

Technology	Version	Role
HDFS / Hadoop	3.3.6	Stores all raw JSON files (~3.7GB across 20 session files)
Apache Spark	3.3.4	Reads HDFS, cleans data, runs analytics, loads MongoDB and HBase
MongoDB	8.x	Stores users, products, categories, transactions
HBase	2.5.15 (Docker)	Stores 100,000 user sessions as time-series with reverse-timestamp keys
Python / PySpark	3.11 / 3.3.4	All scripts, queries, and visualizations

SCREENSHOT: HDFS NameNode UI (localhost:9870) showing/ecommerce/directory with uploaded files



2. Part 1 — Data Modeling and Storage

2.1 MongoDB Schema

MongoDB was chosen for its document model which handles nested e-commerce data naturally. Four collections were created:

Collection	Key Fields	Why MongoDB
users	user_id, geo_data, registration_date	Simple profile documents, quick lookups
products	product_id, category_id, base_price, price_history[]	Embedded price history — no extra table needed
categories	category_id, name, subcategories[]	Full category tree in one document
transactions	transaction_id, user_id, items[], total, payment_method	Embedded items enable powerful aggregation pipelines

We implemented three aggregation pipelines:

- Top 5 products by revenue — prod_00154 led with 343,418 from 281 sales
- Revenue by product category — top category generated 21.7M across 43,106 orders
- Revenue by payment method — gift cards and crypto each generated ~49M

Top 5 products by revenue

```
ecommerce> db.transactions.aggregate([
  { $unwind: "$items" },
  { $group: {
    _id: "$items.product_id",
    total_revenue: { $sum: "$items.subtotal" },
    times_sold: { $sum: 1 }
  }},
  { $sort: { total_revenue: -1 }},
  { $limit: 5 }
])

{ _id: 'prod_00154', total_revenue: 343418.7, times_sold: 281 },
{ _id: 'prod_00253', total_revenue: 332124.24, times_sold: 293 },
{ _id: 'prod_02908', total_revenue: 328168.22, times_sold: 285 },
{ _id: 'prod_01652', total_revenue: 327828.4, times_sold: 290 },
{ _id: 'prod_02303', total_revenue: 326893.25, times_sold: 290 }
```

Revenue by product category

```
{
  _id: 'cat_005',
  total_revenue: 22221952.21,
  total_orders: 40189,
  avg_order_value: 552.9361817910374
},
{
  _id: 'cat_004',
  total_revenue: 21715953.62,
  total_orders: 43106,
  avg_order_value: 503.7803001902288
},
{
  _id: 'cat_014',
  total_revenue: 21434230.580000002,
  total_orders: 43421,
  avg_order_value: 493.63742382718044
},
{
  _id: 'cat_021',
  total_revenue: 20421851.81,
  total_orders: 38384,
  avg_order_value: 532.0407411942475
},
{
  _id: 'cat_019',
  total_revenue: 20265239.61,
  total_orders: 40720,
  avg_order_value: 497.6728784381139
}
```

Revenue by payment method

```
ecommerce> db.transactions.aggregate([
  { $group: {
    _id: "$payment_method",
    total_revenue: { $sum: "$total" },
    num_transactions: { $sum: 1 },
    avg_order_value: { $avg: "$total" }
  }},
  { $sort: { total_revenue: -1 }}
])
[
  {
    _id: 'credit_card',
    total_revenue: 117396199.58,
    num_transactions: 125177,
    avg_order_value: 937.8416129161108
  },
  {
    _id: 'paypal',
    total_revenue: 116678684.82,
    num_transactions: 125185,
    avg_order_value: 932.0500444941486
  },
  {
    _id: 'other',
    total_revenue: 116678684.82,
    num_transactions: 125185,
    avg_order_value: 932.0500444941486
  }
]
```

2.2 HBase Schema

HBase handles our session time-series data. Two tables were designed:

Table 1: user_sessions | Row Key: user_id_reversetimestamp

The reverse timestamp ensures most recent sessions appear first when scanning. All sessions for one user group together under the same prefix — enabling fast O(1) lookups.

Column Family	Columns	Purpose
session_info	session_id, start_time, duration, conversion_status, referrer	Core session details
device	type, os, browser	Device information
geo	city, state, country	Location data
activity	viewed_products, num_page_views, cart_items	User activity

Table 2: product_metrics | Row Key: product_id_date — tracks daily product performance (views, conversions, revenue).

100,000 sessions were loaded into HBase via Apache Spark reading from HDFS, then writing through the happybase Thrift client.

SCREENSHOT: Loading 100,000 sessions into HBase

```
print(f"Done! {count:,} sessions loaded into HBase!")
connection.close()
```

Done! 100,000 sessions loaded into HBase!

SCREENSHOT: HBase shell (creations of user_sessions and product_metrics tables)

```
hbase(main):009:0> create 'user_sessions', {NAME => 'session_info', VERSIONS => 1}, {NAME => 'device', VERSIONS => 1}, {NAME => 'geo', VERSIONS => 1}, {NAME => 'activity', VERSIONS => 1}
Created table user_sessions
Took 5.2856 seconds
=> Hbase::Table - user_sessions
hbase(main):010:0> create 'product_metrics', {NAME => 'stats', VERSIONS => 1}, {NAME => 'inventory', VERSIONS => 1}
Created table product_metrics
Took 1.3015 seconds
=> Hbase::Table - product_metrics
hbase(main):011:0>
```

SCREENSHOT: Retrieving result for a specific user by scanning user_sessions

```
hbase(main):005:0* scan 'user_sessions', {ROWPREFIXFILTER => 'user_000001_', LIMIT => 1}
ROW COLUMN+CELL
user_000001_98218433151999 column=device:os, timestamp=1782604695153, value=iOS
user_000001_98218433151999 column=device:type, timestamp=1782604695153, value=tablet
user_000001_98218433151999 column=geo:city, timestamp=1782604695153, value=Michaelstad
user_000001_98218433151999 column=geo:state, timestamp=1782604695153, value=MO
user_000001_98218433151999 column=session_info:conversion, timestamp=1782604695153, value=browsed
user_000001_98218433151999 column=session_info:duration, timestamp=1782604695153, value=1040
user_000001_98218433151999 column=session_info:referrer, timestamp=1782604695153, value=email
user_000001_98218433151999 column=session_info:session_id, timestamp=1782604695153, value=sess_0ff4363687
user_000001_98218433151999 column=session_info:start_time, timestamp=1782604695153, value=2026-06-16T01:40:48
1 row(s)
```

2.3 MongoDB vs HBase — Why Each Data Type Lives Where

Data	System	Reason
Users, Products, Transactions	MongoDB	Nested documents, complex aggregations, business reporting
Session time-series	HBase	High-volume writes, user-prefix range scans, sparse columns
Product daily metrics	HBase	Append-heavy daily writes, product+date composite key

3. Part 2 — Data Processing with Apache Spark

3.1 Data Cleaning and Normalization

Before any analytics, session data was cleaned in Spark: null rows dropped (`dropna()`), nested structs flattened (`device_profile.type` → `device_type`), DataFrames cached in memory to avoid re-reading HDFS, and only HBase-relevant columns selected.

SCREENSHOT: Loading all data from HDFS and cleaning session dataset code

```
# Load all data
sessions_df = spark.read.json("hdfs://localhost:9000/ecommerce/sessions/sessions_0.json")
transactions_df = spark.read.json("hdfs://localhost:9000/ecommerce/transactions/transactions.json")
users_df = spark.read.json("hdfs://localhost:9000/ecommerce/users/users.json")
products_df = spark.read.json("hdfs://localhost:9000/ecommerce/products/products.json")
categories_df = spark.read.json("hdfs://localhost:9000/ecommerce/categories/categories.json")

# Clean sessions
sessions_clean = sessions_df.select(
    "session_id", "user_id", "start_time", "duration_seconds",
    "conversion_status", "referrer",
    F.col("device_profile.type").alias("device_type"),
    F.col("device_profile.os").alias("device_os"),
    F.col("geo_data.city").alias("city"),
    F.col("geo_data.state").alias("state")
).dropna(subset=["user_id", "session_id", "start_time"])
```

3.2 Spark SQL Analytics

All DataFrames were registered as SQL views and five queries were run:

Query	Key Result
Top 10 Products by Revenue	prod_00154 leads with \$343,418 — top 10 are closely clustered suggesting healthy revenue distribution
Monthly Revenue Trend	Revenue peaked in May at \$160M — March is low because it is a partial month in the dataset
Session Conversion Analysis	20% converted, 44% abandoned, 35% browsed — cart abandonment is the biggest opportunity
Device & OS Usage	All device/OS combinations perform similarly — no single platform dominates
Conversion Rate by Referrer	Search engine leads at 20.71% — users arriving via search are already in buying mode

SCREENSHOT: A Spark query to print top 10 products by Revenue

=== Top 10 Products by Revenue ===

product_id	times_sold	total_units	total_revenue
prod_00154	281	578	343418.7
prod_00253	293	609	332124.24
prod_02908	285	587	328168.22
prod_01652	290	584	327828.4
prod_02303	290	575	326893.25
prod_02419	286	577	324262.46
prod_00967	287	593	323475.57
prod_03324	292	561	323438.94
prod_03061	280	569	318890.36
prod_03278	288	587	315483.15

SCREENSHOT: Spark Query to show Revenue by Month

=== Monthly Revenue Trend ===

month	num_transactions	revenue	avg_order_value
2026-03	30339	2.83552033E7	934.61
2026-04	166766	1.555654936E8	932.84
2026-05	171716	1.6007972427E8	932.24
2026-06	131179	1.2257574205E8	934.42

SCREENSHOT: Query for Session conversion Analysis

=== Session Conversion Analysis ===

conversion_status	num_sessions	percentage	avg_duration_minutes
abandoned	44700	44.70	30.25
browsed	35061	35.06	30.18
converted	20239	20.24	30.08

SCREENSHOT: QQuery for Device Usage Analysis

```
=== Device Usage Analysis ===
```

device_type	device_os	num_sessions	avg_duration_min	conversions
tablet	macOS	8490	30.53	1725
desktop	Windows	8400	29.8	1637
mobile	macOS	8374	30.0	1658
desktop	Android	8374	29.96	1742
desktop	iOS	8352	30.39	1711
mobile	iOS	8346	29.99	1728
tablet	iOS	8346	30.28	1672
desktop	macOS	8310	30.39	1706
tablet	Android	8309	30.19	1690
mobile	Windows	8276	29.99	1641
tablet	Windows	8225	30.3	1665
mobile	Android	8198	30.45	1664

SCREENSHOT: Query for User Segmentation by Referrer

```
=== Sessions by Referrer Source ===
```

referrer	num_sessions	conversions	conversion_rate
direct	20094	4095	20.38
email	20066	3918	19.53
affiliate	20027	4111	20.53
social	19989	4010	20.06
search_engine	19824	4105	20.71

3.3 Product Recommendation Indicators

We implemented a 'users who bought X also bought Y' recommendation engine using a self-join on the transactions DataFrame. Products appearing in the same transaction are identified as co-purchased pairs:

Product A	Product B	Times Bought Together
prod_02483	prod_02898	5
prod_03836	prod_04523	4
prod_01527	prod_04212	4

When a user adds prod_02483 to their cart, the system recommends prod_02898. Low counts here are due to using a data subset — the full 2M sessions would produce much stronger signals.

SCREENSHOT: Product recommendation query result

```
=== Product Recommendation Indicators ===
+-----+-----+-----+
| product_a | product_b | times_bought_together |
+-----+-----+-----+
| prod_02483 | prod_02898 | 5 |
| prod_03836 | prod_04523 | 4 |
| prod_01527 | prod_04212 | 4 |
| prod_01677 | prod_04867 | 4 |
| prod_00297 | prod_03662 | 4 |
| prod_00335 | prod_04369 | 4 |
| prod_00950 | prod_01968 | 3 |
| prod_00993 | prod_04785 | 3 |
| prod_02365 | prod_04308 | 3 |
| prod_01652 | prod_03882 | 3 |
+-----+-----+-----+
```

4. Part 3 — Analytics Integration: Customer Lifetime Value

Business Question: Who are our most valuable customers and what makes them valuable?

This query combines three data sources through Spark:

- Step 1 — MongoDB (transactions): aggregate total spend, order count, avg order value per user
- Step 2 — HBase (sessions): aggregate total sessions and avg session duration per user
- Step 3 — Spark JOIN: combine users + transactions + sessions → calculate CLV score

CLV Formula: $\text{estimated_clv} = \text{total_spend} \times (1 + \text{total_sessions} \times 0.01)$

This rewards both high spenders AND highly engaged users.

User ID	State	Orders	Total Spend (\$)	Sessions	Est. CLV (\$)
user_004099	NJ	74	79,297.58	17	92,778.17
user_001824	AZ	74	78,556.86	15	90,340.39
user_001840	MS	79	76,244.69	15	87,681.39
user_005418	VI	72	75,542.10	16	87,628.84
user_005142	MO	73	80,881.92	8	87,352.47

Note: user_004099 ranks #1 despite not having the highest raw spend — their 17 sessions push their CLV higher, reflecting stronger long-term loyalty.

SCREENSHOT: CLV query result showing top 10 customers

user_id	state	total_orders	total_spend	avg_order_value	total_sessions	avg_session_minutes	estimated_clv
user_004099	NJ	74	79297.58	1071.59	17	31.55	92778.17
user_001824	AZ	74	78556.86	1061.58	15	35.1	90340.39
user_001840	MS	79	76244.69	965.12	15	24.98	87681.39
user_005418	VI	72	75542.1	1049.2	16	30.25	87628.84
user_005142	MO	73	80881.92	1107.97	8	29.39	87352.47
user_003511	MO	62	75023.48	1210.06	16	36.19	87027.24
user_005319	ID	80	75450.06	943.13	14	19.99	86013.07
user_003411	VT	71	76684.48	1080.06	10	37.01	84352.93
user_001827	KY	63	74579.47	1183.8	13	31.48	84274.8
user_000720	OK	72	76468.51	1062.06	10	31.43	84115.36

5. Part 4 — Visualizations and Business Insights

Four charts were generated using Matplotlib and Seaborn from Spark SQL results:

SCREENSHOT: Some analytics dashboard

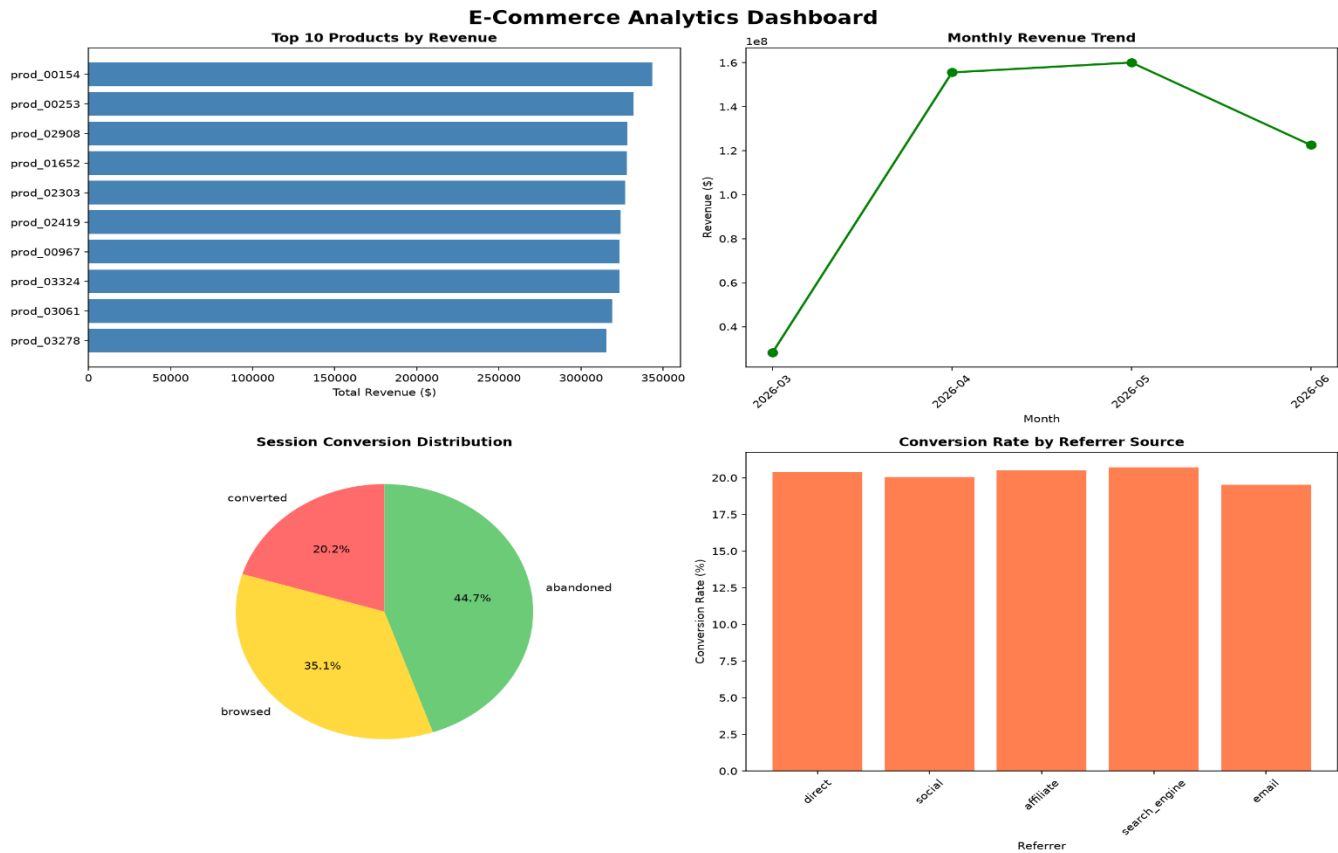


Chart	What It Shows	Key Insight
Top 10 Products (bar)	Revenue per product	Top 10 are closely clustered — healthy, no single product dominates
Monthly Revenue (line)	Revenue trend over time	April-May peak at 155M-160M; plateau suggests stable trading
Conversion Pie	Browsed vs abandoned vs converted	44% cart abandonment — biggest opportunity for revenue recovery
Referrer Bar	Conversion rate by traffic source	Search engine converts best (20.71%); email lowest (19.53%)

6. Scalability, Limitations and Future Work

Scalability

Component	Current Setup	Production Approach
HDFS	1 NameNode, 1 DataNode	Add DataNodes; increase replication to 3
HBase	Single Docker container	Add RegionServers to distribute row ranges
MongoDB	Single node	Enable sharding on user_id
Spark	local[2], 2GB RAM	Deploy on YARN cluster with multiple executors

Limitations

- RAM constraints (8GB at 87-97% usage) limited HBase to 100K sessions instead of the full 2M
- HBase on Windows via Docker adds overhead — Thrift server required manual restarts
- PySpark version mismatch (3.5.1 vs Spark JVM 3.3.4) required downgrade before Spark SQL worked
- Product co-purchase counts are low (max 5) due to using a data subset

Future Work

- Load all 20 session files on a proper cluster (16GB+ RAM)
- Build a collaborative filtering model using Spark MLlib ALS
- Implement Spark Structured Streaming for real-time HBase ingestion from Kafka
- Add an interactive dashboard using Apache Superset

7. Conclusion

This project shows that the right tool for each job matters more than finding one tool that does everything. MongoDB handled nested business documents with powerful aggregation pipelines. HBase delivered fast time-series session lookups through smart row key design. Spark tied everything together — processing 500K transactions, running complex SQL, and joining data across systems that would never talk to each other otherwise.

The results are real and actionable: a 44% cart abandonment rate representing millions in recoverable revenue, a clear picture of our top 10 most valuable customers, product pairs ready to power a recommendation engine, and a monthly revenue trend showing where the business peaks. That is what big data analytics is actually for.